

Operating Systems Tutorial

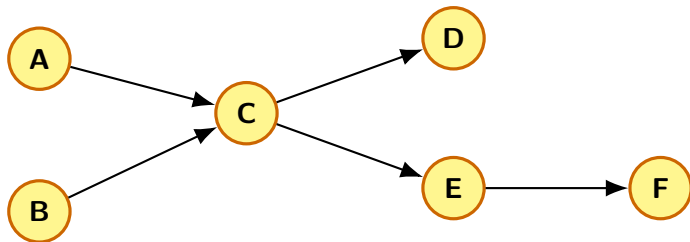
Semaphore Synchronization Exercises

Exercise 1: Ordering Constraints

A process has two concurrently executing threads, T_1 and T_2 . The six operations A, B, C, D, E, F are distributed as follows:

- T_1 executes A, E, F .
- T_2 executes B, C, D .

The required ordering constraints are: C must execute after both A and B finish; D and E must execute after C finishes; F must execute after E finishes.



Use semaphore `wait()` and `signal()` operations to describe the synchronization between T_1 and T_2 , and state the meaning and initial value of each semaphore.

Answer 1: Cross-Thread Synchronization

Only two cross-thread dependencies need semaphores: $A \rightarrow C$ and $C \rightarrow E$. The other dependencies are already enforced by local program order.

```
semaphore a_done = 0; // A has completed
semaphore c_done = 0; // C has completed

// Thread T1
A;
signal(a_done);

wait(c_done);
E;
F;

// Thread T2
B;
wait(a_done);
C;
signal(c_done);
D;
```

- $a_done = 0$: blocks C until A has finished.
- $c_done = 0$: blocks E until C has finished.
- $B \rightarrow C$, $C \rightarrow D$, and $E \rightarrow F$ are guaranteed by the sequential execution inside each thread.

Exercise 2: Mutual Exclusion with Swap

Students are asked to implement mutual exclusion for a critical section using a hardware swap instruction and a shared Boolean variable `lock`.

- `lock == TRUE`: a thread must not enter the critical section.
- `lock == FALSE`: a thread may enter the critical section.

A student's pseudo-code is:

```
bool lock = FALSE; // shared variable
...
bool key = TRUE;
if (key == TRUE)
    swap(key, lock);

critical_section();

lock = TRUE;
...
```

A proposed function replacement is:

```
void newSwap(bool *a, bool *b)
{
    bool temp = *a;
    *a = *b;
    *b = temp;
}
```

Questions: identify the wrong statements and correct them without adding statements. Can `newSwap(&key, &lock)` replace the hardware swap?

Answer 2: Corrected Swap Protocol

The two wrong statements are the entry test and the exit assignment.

```
bool lock = FALSE; // shared variable; FALSE means available
...
bool key = TRUE;
while (key == TRUE)
    swap(key, lock); // atomic exchange

critical_section();

lock = FALSE;
...
```

- if must be changed to while; otherwise a thread enters even when the swap leaves `key == TRUE`.
- The exit statement must set `lock = FALSE`, because FALSE means the critical section is available.

Answer 2: Why newSwap Is Not Enough

`newSwap(&key, &lock)` cannot replace the hardware `swap` instruction.

The reason is atomicity:

- A hardware `swap` is one indivisible operation.
- `newSwap` is compiled into several loads and stores.
- Two threads can interleave those loads and stores and both observe `lock == FALSE`.

Example bad interleaving:

$$T_1 : *a = *b \text{ (FALSE)} \quad T_2 : *a = *b \text{ (FALSE)}$$

Both local `key` variables may become `FALSE`, so both threads can leave the loop and enter the critical section. Mutual exclusion is lost.

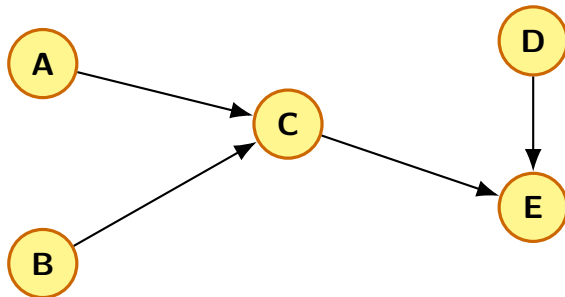
Exercise 3: Partial Ordering of Five Operations

There are five operations A, B, C, D, E .

Required ordering constraints:

- C must execute after both A and B finish.
- E must execute after both C and D finish.

Use semaphore `wait()` and `signal()` operations to describe the synchronization among these operations, and state the meaning and initial value of each semaphore.



Answer 3: Counting Semaphores

Use one counting semaphore for each join point.

```
semaphore before_C = 0; // completed predecessors of C: A and B
semaphore before_E = 0; // completed predecessors of E: C and D

// Operation A
A;
signal(before_C);

// Operation B
B;
signal(before_C);

// Operation C
wait(before_C);
wait(before_C);
C;
signal(before_E);

// Operation D
D;
signal(before_E);

// Operation E
wait(before_E);
wait(before_E);
E;
```

Each semaphore starts at 0; each predecessor contributes one `signal()`, and the dependent operation consumes all required signals.

Exercise 4: Philosophers with Bowls and Chopsticks

There are n philosophers, $n \geq 3$, sitting around a round table. They alternate between eating and thinking.

- There are m bowls in the center, $m \geq 1$.
- There is one chopstick between every two neighboring philosophers.
- A philosopher must obtain one bowl and the two adjacent chopsticks before eating.
- After eating, the philosopher returns the bowl and both chopsticks, then continues thinking.

Use semaphore `wait()` and `signal()` operations to maximize the number of philosophers that can eat simultaneously while preventing deadlock. State the meaning and initial value of each semaphore.

Answer 4: Deadlock-Free Resource Ordering

Number the chopsticks $0, 1, \dots, n - 1$. Philosopher i 's adjacent chopsticks are i and $(i + 1) \bmod n$.

```
semaphore bowl = m;           // available bowls
semaphore chopstick[n] = {1, ..., 1}; // each chopstick is a mutex

// Philosopher i
while (TRUE) {
    think();

    int left = i;
    int right = (i + 1) % n;
    int first = min(left, right);
    int second = max(left, right);

    wait(chopstick[first]);
    wait(chopstick[second]);
    wait(bowl);

    eat();

    signal(bowl);
    signal(chopstick[second]);
    signal(chopstick[first]);
}
```

Answer 4: Semaphore Meanings

- `chopstick[i] = 1`: binary semaphore protecting chopstick i .
- `bowl = m`: counting semaphore for the m available bowls.
- All philosophers acquire chopsticks in increasing chopstick-number order, so circular wait on chopsticks is impossible.
- A bowl is acquired only after both chopsticks have been obtained, so bowls are not occupied by philosophers who cannot yet eat.

Therefore the algorithm can reach the physical maximum number of simultaneous eaters:

$$\min \left(m, \left\lfloor \frac{n}{2} \right\rfloor \right).$$

Deadlock is prevented; starvation freedom additionally requires fair semaphore queues.